Formação Completa

AWS Developer Associate

Objetivos da Aula

Comparar políticas AWS

Entender AWS Managed Policies e Customer-Managed Policies para implementar o Princípio do Menor Privilégio.

Configurar acesso seguro

Implementar acesso programático seguro e aplicar o modelo RBAC.

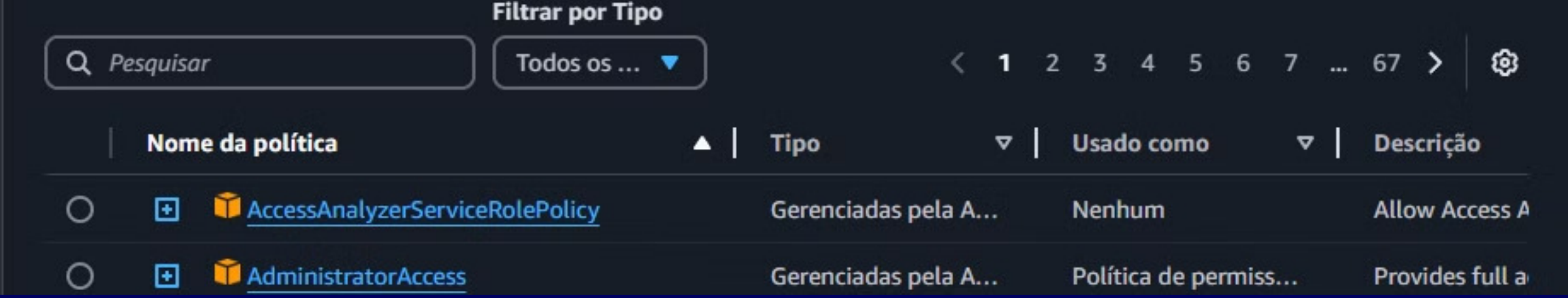
Implementar criptografia

Proteger dados em repouso e em trânsito com estratégias de criptografia.

Gerenciar chaves e certificados

Utilizar AWS KMS, CloudTrail, ACM e CloudHSM.





AWS Managed Policies

Definição

Conjuntos de permissões predefinidos e gerenciados pela AWS com base em boas práticas de segurança.

Vantagens

Facilitam configuração inicial e manutenção. Garantem conformidade com melhores práticas sem ajustes manuais.

Desvantagens

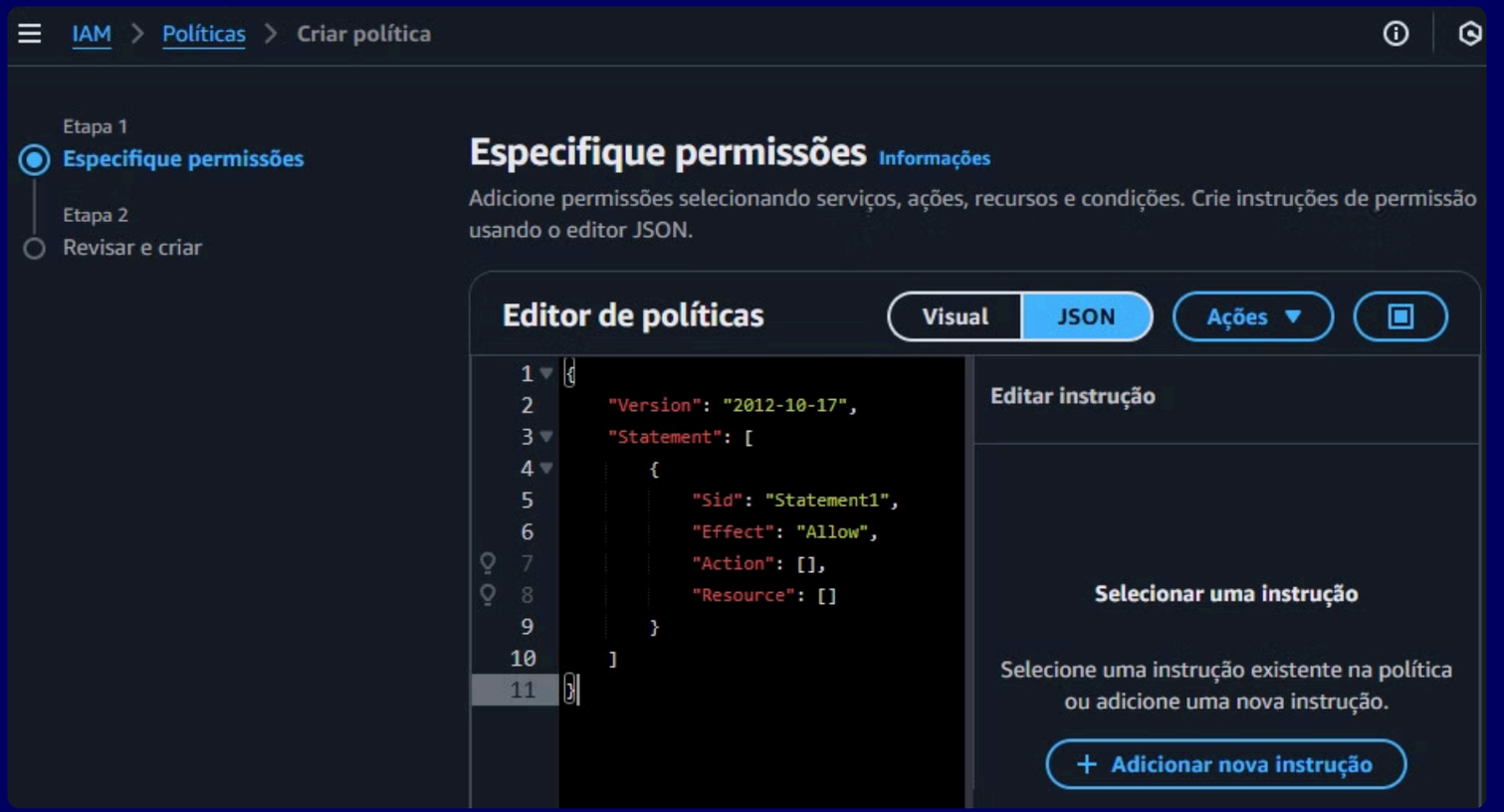
Menor flexibilidade para personalização. Podem conceder mais permissões que o necessário.

Exemplo de AWS Managed Policy

```
aws iam attach-group-policy \
  --group-name MeuGrupo \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

- Este comando anexa a política gerenciada AmazonS3ReadOnlyAccess ao grupo MeuGrupo, concedendo permissões de leitura para o S3.
- Lembre-se: Boas para início rápido e funções padrão, mas sempre valide se as permissões são adequadas ao Princípio do Menor Privilégio.

Customer-Managed Policies



Definição

Políticas criadas e mantidas por você, permitindo personalização total das permissões para necessidades específicas.

Vantagens

Ajuste fino das permissões. Ideal para aplicar estritamente o Princípio do Menor Privilégio.

Desvantagens

Exigem manutenção contínua e conhecimento aprofundado de segurança para evitar configurações incorretas.

Exemplo de Customer-Managed Policy definida via AWS CLI

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::meu-bucket",
        "arn:aws:s3:::meu-bucket/*"
      ]
    }
  ]
}
```

Esta política permite apenas leitura em um bucket S3 específico, aplicando o Princípio do Menor Privilégio.

Comparação entre Tipos de Políticas

AWS Managed Policies

- Criadas e mantidas pela AWS
- Atualizações automáticas
- Menor controle granular
- Ideal para casos de uso comuns
- Configuração rápida

Customer-Managed Policies

- Criadas e mantidas por você
- Atualizações manuais
- Controle granular total
- Ideal para requisitos específicos
- Aplicação estrita do PoLP **(Principle of Least Privilege)**

O que é o Role-Based Access Control (RBAC) na AWS?

O **Role-Based Access Control (RBAC)** é um modelo de controle de acesso baseado em funções, amplamente utilizado para gerenciar permissões em sistemas de TI. Na **AWS**, o RBAC é implementado por meio do **AWS Identity and Access Management (IAM)**, permitindo que usuários e serviços obtenham permissões de acordo com suas funções dentro da organização.

Elementos-chave do RBAC na AWS:

1 IAM Roles

Entidades com permissões específicas que podem ser assumidas por usuários, serviços AWS ou identidades externas.

Exemplo: Uma função S3ReadOnlyAccess permite apenas leitura de objetos no S3.

2 IAM Policies

Documentos JSON que definem permissões precisas e são anexados às funções.

Controlam quais ações são permitidas ou negadas em recursos específicos da AWS.

3 Usuários e Grupos

Identidades que podem assumir funções para obter permissões temporárias.

Simplifica o gerenciamento ao atribuir permissões por função, não por indivíduo.

4 Assunção de Funções (Role Assumption)

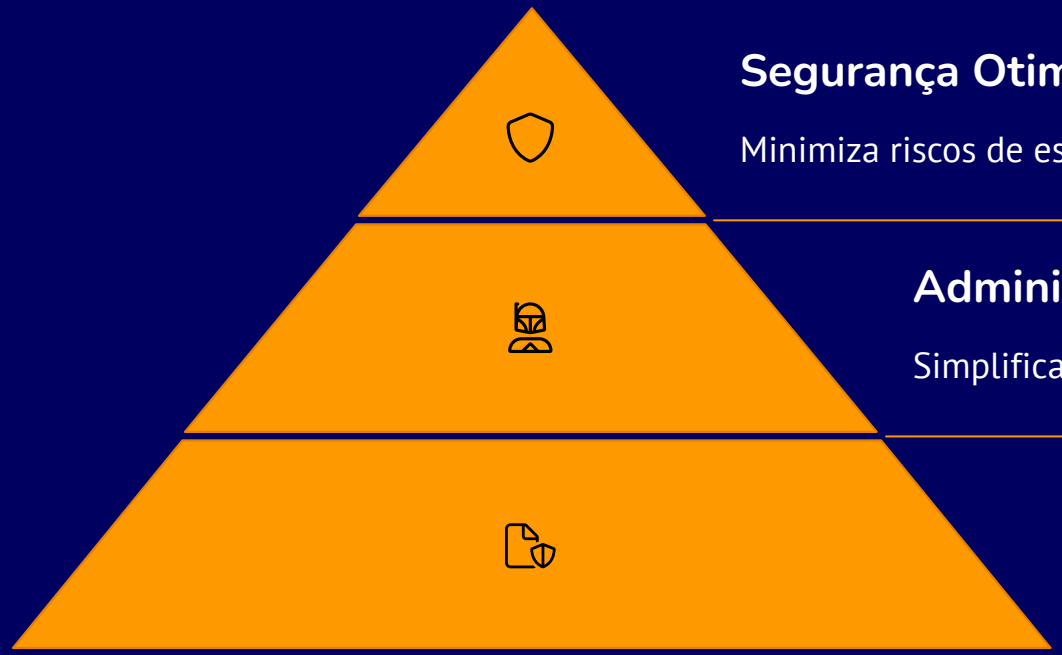
Processo realizado via AWS STS para obter credenciais temporárias.

Melhora a segurança limitando o tempo de acesso aos recursos.

O RBAC na AWS simplifica o gerenciamento de acessos, fortalece a segurança e facilita auditorias ao conceder permissões baseadas em funções organizacionais.

Role-Based Access Control (RBAC)

Benefícios



Segurança Otimizada

Minimiza riscos de escalonamento de privilégios

Administração Centralizada

Simplifica gerenciamento de permissões

Princípio do Menor Privilégio

Permissões baseadas em funções específicas

RBAC garante acesso com base nas funções, reduzindo riscos e facilitando a gestão.zação.

Exemplo de RBAC com Python

```python
import boto3

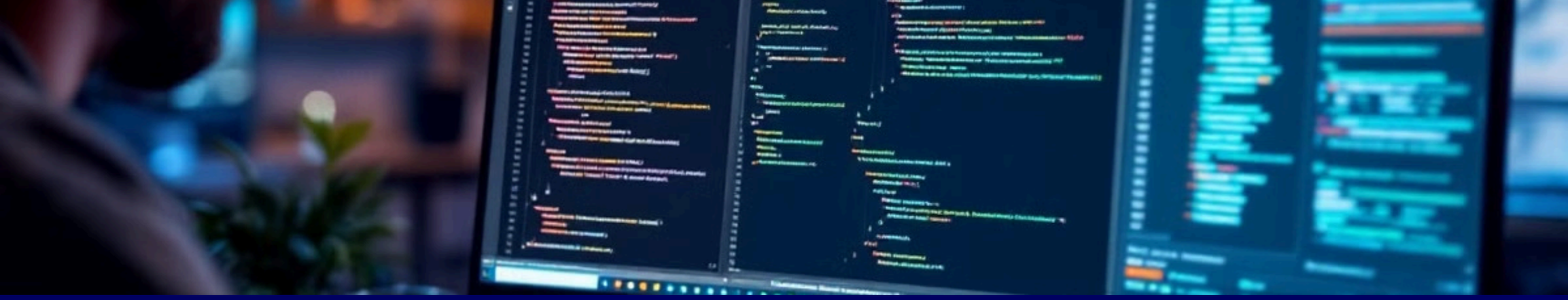
# Cliente S3 com permiss es restritas definidas via IAM
s3_client = boto3.client('s3', region_name='us-east-1')

def listar_buckets_restritos():
    try:
        response = s3_client.list_buckets()
        return [bucket['Name'] for bucket in response.get('Buckets', [])]
    except Exception as e:
        print("Erro ao listar buckets:", e)
        return None

print("Buckets acessíveis:", listar_buckets_restritos())
```

Esse cenário ocorre ao conceder acesso restrito para listar apenas os buckets do S3, seguindo o princípio do menor privilégio e o modelo **RBAC** (Role-Based Access Control). A restrição é aplicada no **IAM** com uma política que permite apenas **s3:ListBucket**. O código usa as permissões concedidas.

- **boto3.client('s3')**, assume que as credenciais do IAM já estão configuradas.



Acesso Programático

Definição

Configuração de credenciais que permitem aplicações, scripts e ferramentas interagirem com serviços AWS via APIs.

Implementação

Utiliza Chaves de Acesso para usuários IAM ou credenciais temporárias via AWS STS para Roles.

Segurança

Aplique o Princípio do Menor Privilégio. Evite chaves de longa duração.

Automação

Fundamental para automação, IaC e integração de sistemas.

- As credenciais devem estar configuradas via IAM (como em **~/.aws/credentials** ou variáveis de ambiente).

Por que o uso do acesso programático com RBAC é importante?

1. **Segurança** Permite aplicar o princípio do menor privilégio, evitando acessos desnecessários.
2. **Automação segura** Facilita o uso de scripts, pipelines e aplicações com permissões controladas, sem expor credenciais.
3. **Escalabilidade** Permissões são atribuídas por função, o que simplifica a gestão em ambientes grandes.
4. **Auditoria** Facilita o rastreamento de ações por papéis, melhorando a visibilidade e o controle.

Por que o que vimos sobre Políticas do IAM é importante? (Visão Prática e da Certificação)

Importância Prática, Certificação e Mercado de Trabalho

- O princípio do menor privilégio é essencial para **segurança e conformidade** em ambientes corporativos.
- Na certificação AWS Certified Developer – Associate, é fundamental demonstrar controle adequado de acessos.
- No mercado, empresas valorizam profissionais que sabem configurar acessos seguros, reduzindo riscos e facilitando auditorias.

Escolha Adequada de Políticas

- **AWS Managed Policies**: Práticas recomendadas da AWS, facilitam gestão e manutenção.
- **Customer-Managed Policies**: Permitem personalização para atender necessidades específicas da empresa.
- Profissionais que sabem equilibrar **facilidade de gerenciamento e segurança** são altamente demandados.

Configuração de Acesso Programático e RBAC

- Empresas esperam que desenvolvedores consigam configurar **chaves de acesso seguras** para aplicações.
- O modelo **RBAC** garante que apenas usuários certos tenham permissões necessárias, evitando exposição de dados.
- Saber **monitorar e revisar acessos regularmente** é um diferencial competitivo no mercado.

Criptografia na AWS

Objetivos

Capacitar você a aplicar criptografia para proteger dados em repouso e em trânsito.

Compreender a gestão, rotação e proteção de chaves na AWS.

Integrar mecanismos de criptografia com outros serviços AWS.

Resultados Esperados

Implementar criptografia para dados em repouso e em trânsito.

Configurar e gerenciar chaves de criptografia no AWS KMS.

Integrar soluções de criptografia em suas aplicações.



Criptografia em Repouso

Definição

Protege dados armazenados em discos (EBS), storage (S3), bancos de dados (RDS, DynamoDB) e outros sistemas.

Funcionamento

Utiliza algoritmos como AES-256 e chaves gerenciadas para cifrar dados antes da gravação física.

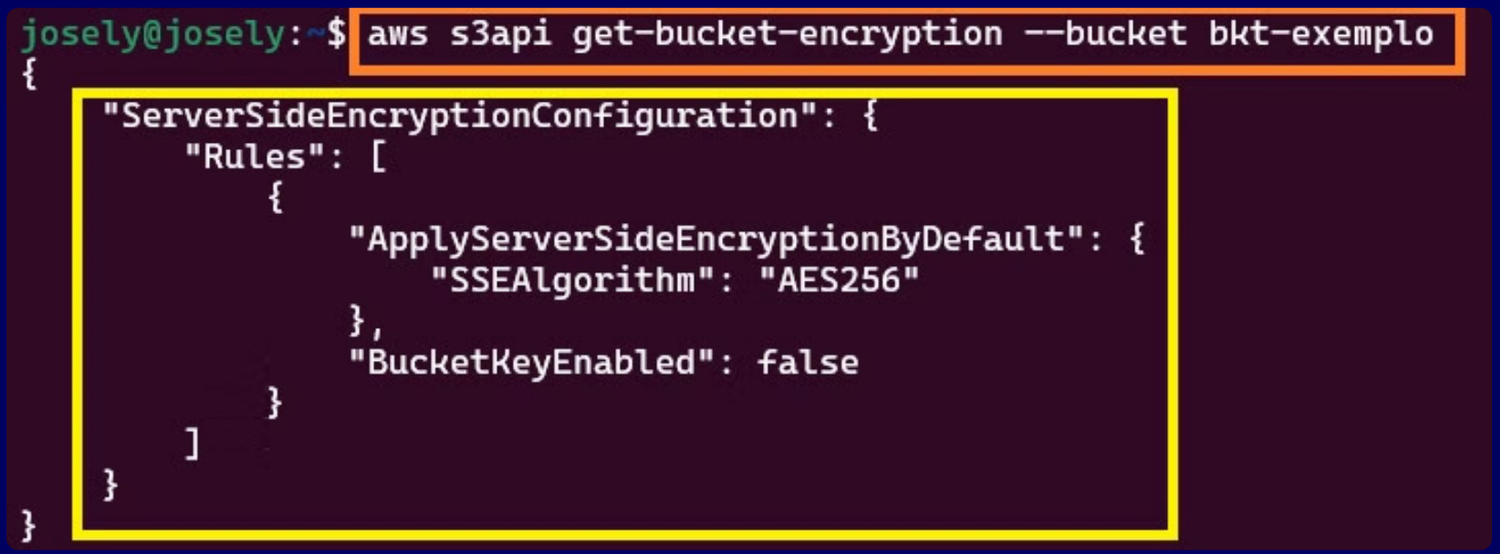
Importância

Crucial para proteger informações sensíveis e cumprir requisitos de conformidade (LGPD, GDPR, HIPAA).

Verificando Criptografia em Bucket S3

aws s3api get-bucket-encryption --bucket bkt-exemplo

Este comando verifica a configuração de criptografia padrão de um bucket S3:



ApplyServerSideEncryptionByDefault: Esta seção indica que existe uma regra de criptografia padrão configurada para o bucket bkt-exemplo.

"SSEAlgorithm": "AES256": Confirma que novos objetos serão criptografados com AES256 (SSE-S3) por padrão pela AWS.

"BucketKeyEnabled": false: O recurso S3 Bucket Key está desabilitado. A Chave de Bucket S3 pode ajudar a reduzir os custos de solicitação ao KMS, mas não a criptografia em si.



Criptografia em Trânsito

Definição

Assegura que dados transferidos entre sistemas sejam protegidos contra interceptação durante a transmissão.

Funcionamento

Utiliza protocolos seguros como TLS/SSL para estabelecer um canal seguro e cifrar os dados em trânsito.

Importância

Vital para proteger informações sensíveis na rede. Essencial para comunicações seguras na internet.

Exemplo em Python utilizando a biblioteca requests para realizar uma conexão HTTPS com verificação de certificado:

```python
import requests

def fetch_secure_data(url):
    try:
        # A requisição HTTPS garante que os dados sejam transmitidos de forma segura
        response = requests.get(url, timeout=5)
        if response.status_code == 200:
            return response.json()
        else:
            print("Erro na requisição:", response.status_code)
    except requests.exceptions.RequestException as e:
        print("Exceção durante a requisição:", e)
        return None

data = fetch_secure_data("https://api.exemplo.com/dados")
print("Dados obtidos com segurança:", data)
```

AWS KMS: Amazon Key Management Service, que permite a criação, gerenciamento e rotação de chaves de criptografia.

Este código Python realiza uma conexão HTTPS com verificação de certificado, garantindo criptografia em trânsito.

AWS Key Management Service (KMS)

Serviço gerenciado para criação, armazenamento, rotação e controle de acesso a chaves de criptografia.

Tipos de chaves: gerenciadas pela AWS ou personalizadas (customer-managed keys).

Integração automática: funciona com vários serviços da AWS para cifrar dados.

Princípio do menor privilégio: restringe o acesso apenas a identidades autorizadas.

Benefícios: melhora a segurança e simplifica a administração.

Requisitos: controle rigoroso das políticas de acesso e monitoramento constante.



O AWS KMS facilita a criação, armazenamento, gerenciamento e controle de chaves de criptografia.

Exemplo de Uso do KMS

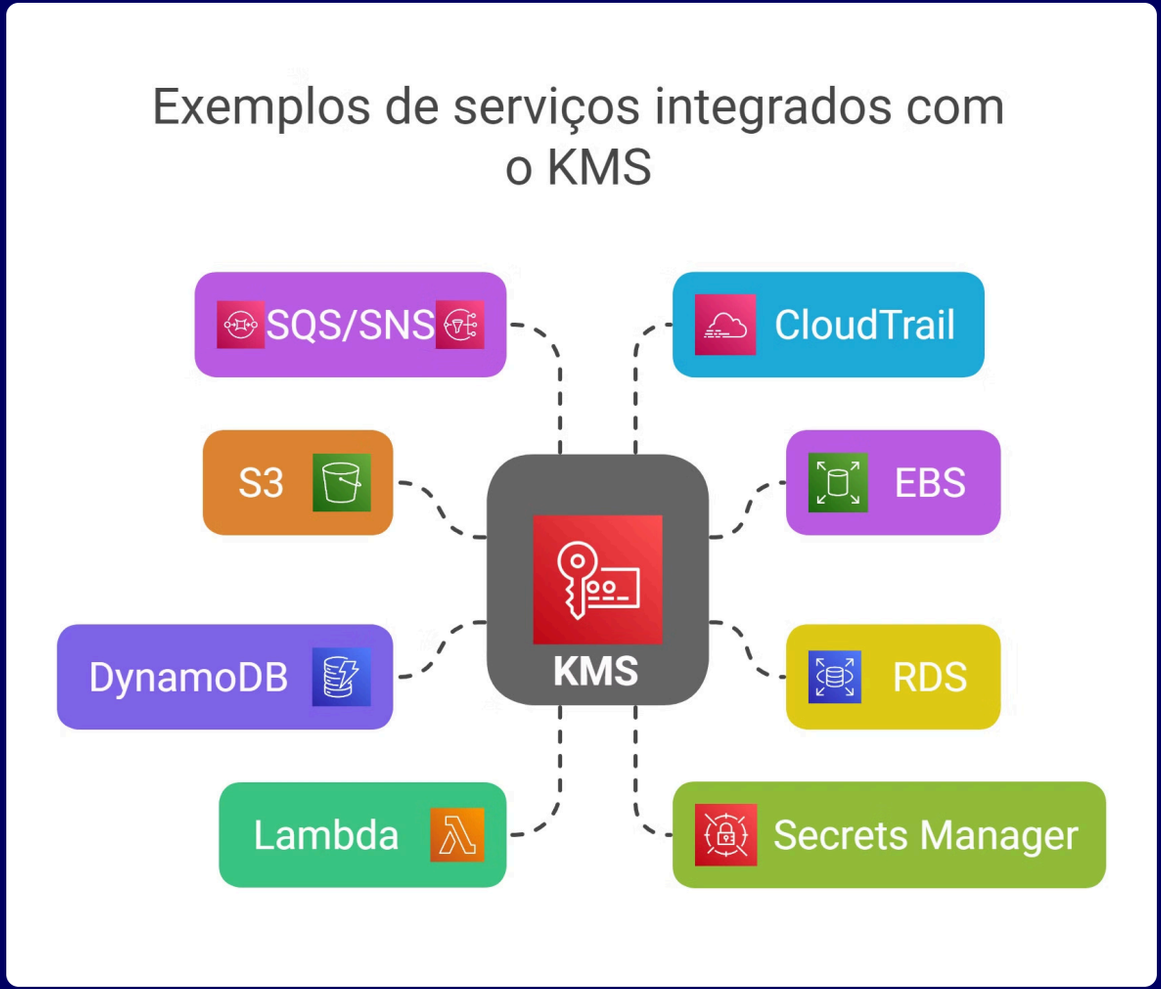
```python
import boto3

# Cria um cliente KMS
kms = boto3.client('kms')

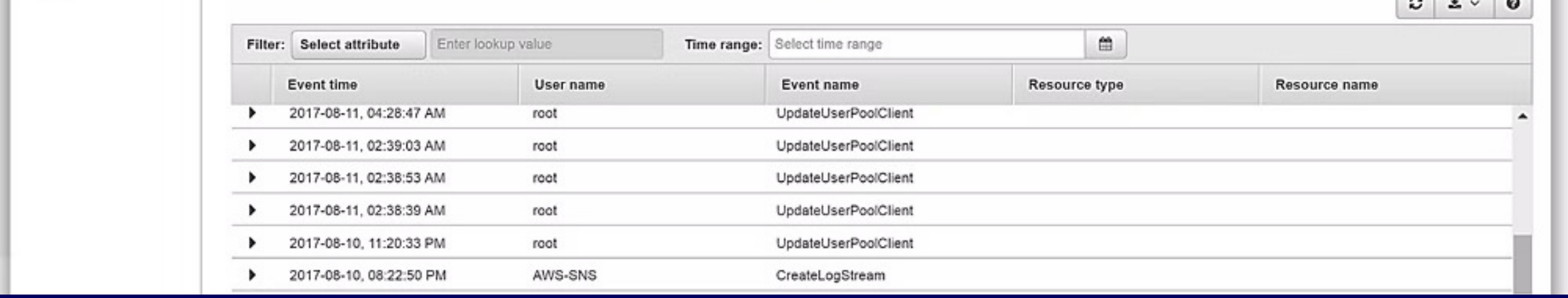
# Cria uma chave gerenciada pelo cliente
response = kms.create_key(Description='Minha chave de exemplo')
key_id = response['KeyMetadata']['KeyId']
print("Chave criada:", key_id)

# Exemplo de cifragem de dados
plaintext = b"Dados sensíveis"
encrypted = kms.encrypt(KeyId=key_id, Plaintext=plaintext)['CiphertextBlob']
print("Dados cifrados:", encrypted)
```

Integração Nativa do KMS com Serviços AWS



O KMS integra-se nativamente com dezenas de serviços AWS para criptografia em repouso transparente.



Auditoria de Chaves KMS com CloudTrail

Operação KMS

Uso de chaves para criptografia/descriptografia

Registro CloudTrail

Todas as chamadas com chaves criptográficas são registradas

Análise

Monitoramento e auditoria dos eventos

Segurança

Detecção de uso não autorizado

Exemplo de Auditoria com CloudTrail

```bash
#!/bin/bash

# ============================
# Script para consultar eventos do CloudTrail relacionados a uma chave KMS específica
# ============================

# Substitua pelo ARN da sua chave KMS
KEY_IDENTIFIER="arn:aws:kms:us-east-1:123456789012:key/EXAMPLE-KEY-ID"

# Região da AWS onde a chave está localizada
REGION="us-east-1"

# Comando para buscar os eventos no CloudTrail relacionados à chave
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=ResourceName,AttributeValue=$KEY_IDENTIFIER \
  --region $REGION \
  --query 'Events[*].{EventTime:EventTime, EventName:EventName, Username:Username}' \
  --output table
```

Este comando retorna os eventos que envolvem a chave especificada, permitindo auditoria detalhada do uso.

Exemplo de Auditoria com CloudTrail: Saída esperada

EventTime	EventName	Username
2025-04-21T14:32:17Z	Encrypt	user1@example.com
2025-04-21T14:35:22Z	Decrypt	lambda-role-abc
2025-04-21T15:10:45Z	GenerateDataKey	ec2-instance-role-xyz
2025-04-21T16:01:13Z	Decrypt	user2@example.com
2025-04-21T16:05:27Z	Encrypt	myapp-service-role

AWS Certificate Manager (ACM)



Definição

Serviço que simplifica o provisionamento e gerenciamento de certificados SSL/TLS para serviços AWS.

Renovação Automática

Gerencia o processo de renovação para certificados públicos, reduzindo o risco de expiração.

Certificados Gratuitos

Emite certificados públicos validados por domínio sem custo adicional.

Integração Nativa

Integra-se com ELB, CloudFront, API Gateway e outros serviços AWS.

Exemplo de Solicitação de Certificado

```
aws acm request-certificate \
  --domain-name www.exemplo.com \
  --validation-method DNS
```

Este comando solicita um certificado SSL/TLS para um domínio via AWS CLI.

Após a solicitação, o ACM fornecerá registros CNAME para validação do domínio.



AWS CloudHSM

Definição

Serviço que fornece módulos de segurança de hardware (HSMs) dedicados, single-tenant, na nuvem AWS.

Casos de Uso

Requisitos regulatórios rigorosos. Controle total sobre chaves. Operações criptográficas de alta performance.

Comparado ao KMS

Maior controle e segurança, mas com maior complexidade e custo. KMS é mais simples e integrado.



Exemplo de Uso do CloudHSM

```
aws cloudhsmv2 describe-clusters --region us-east-1
```

Este comando lista clusters CloudHSM existentes via AWS CLI.

A configuração real envolve criar VPC, subnets, security groups, o cluster HSM, inicializá-lo e criar usuários.

O CloudHSM fornece HSMs dedicados para gerenciamento de chaves com controle total, ideal para compliance rigoroso.



Comparação: KMS vs CloudHSM

AWS KMS

Serviço gerenciado multi-tenant

Fácil integração com serviços AWS

Menor custo operacional

AWS tem acesso limitado às chaves

FIPS 140-2 Nível 2

Ideal para maioria dos casos

AWS CloudHSM

Hardware dedicado single-tenant

Integração mais complexa

Maior custo operacional

AWS não tem acesso às chaves

FIPS 140-2 Nível 3

Para requisitos regulatórios estritos

Estratégias de Criptografia na AWS

1

Nível Básico

Criptografia padrão em serviços AWS

2

Nível Intermediário

KMS com chaves gerenciadas pelo cliente

3

Nível Avançado

CloudHSM para controle total

Escolha a estratégia de criptografia adequada com base nos requisitos de segurança, conformidade e controle da sua organização.



Melhores Práticas de Segurança AWS

Implementar Princípio do Menor Privilégio

Use Customer-Managed Policies específicas e RBAC.

Criptografar Dados Sensíveis

Utilize KMS para dados em repouso e TLS para dados em trânsito.

Rotacionar Credenciais e Chaves

Configure rotação automática quando possível.

Monitorar e Auditar

Use CloudTrail para registrar todas as atividades.

Casos de Uso de Segurança AWS



Serviços Financeiros

Bancos utilizam CloudHSM para proteção de dados sensíveis e conformidade com regulamentações do setor financeiro.



Saúde

Hospitais implementam KMS para criptografia de dados de pacientes, garantindo conformidade com HIPAA.



E-commerce

Lojas online usam ACM para certificados SSL/TLS e KMS para proteger dados de pagamento.

Por que isso é Importante ?

(Visão Prática e da Certificação)

Proteção dos Dados

A criptografia é essencial para proteger dados em repouso e em trânsito, garantindo segurança mesmo em caso de acesso indevido.

Destaques:

- Garante confidencialidade e integridade dos dados.
- Protege contra acessos não autorizados, mesmo com falhas físicas.
- Facilita a escalabilidade segura de sistemas.

Na certificação:

- É esperado que o desenvolvedor saiba aplicar essas práticas em diferentes contextos de aplicação.

Gestão de Chaves e Integração com Serviços

O uso correto do AWS KMS é fundamental para controlar o acesso a dados sensíveis e automatizar a rotação de chaves.

Destaques:

- Centraliza a gestão de chaves e facilita o controle de acesso.
- Integra-se com serviços como ACM e CloudHSM para segurança completa/complexa.
- Reduz riscos de exposição de credenciais.

Na certificação:

- É importante demonstrar conhecimento sobre criptografia e integração com os serviços de segurança da AWS.